

Daily Climate Big Data Pipeline

Büyük Veri Analizine Giriş

Adem Alperen Arda – 220201048

Ubeydullah Gür – 220201079

Barış Cerit – 220201027

1. Proje Mimarisi

1.1 Genel Tasarım

Bu projede günlük iklim verilerini kullanarak baştan sona çalışan bir büyük veri akışı kurduk. Amacımız, Kaggle'dan gelen parquet formatındaki veriyi Kafka üzerinden akıtmak, Spark ile işlemek, Delta Lake katmanlarına yazmak ve en sonunda bu veriyi makine öğrenmesi ile dashboard tarafında kullanmaktı. Yani sadece modeli eğitmek değil, modelin kullanacağı verinin nasıl hazırlandığını da uygulamalı şekilde görmüş olduk.

Sistemi Docker container'ları ile kurduğumuz için her servis kendi ortamında çalıştı. Bu da projeyi farklı bilgisayarlarda daha kolay çalıştırılabilir hale getirdi. Projedeki genel akış aşağıdaki gibi ilerledi:

Kaggle Parquet -> Kafka Producer -> Kafka Topic -> Spark Streaming -> Bronze / Silver / Gold -> ML Modelleri + Dashboard

1.2 Medallion Katmanları

Verileri tek bir yerde karışık şekilde tutmak yerine Bronze, Silver ve Gold olarak üç katmana ayırdık. Bu yaklaşım sayesinde ham veri, temizlenmiş veri ve analize hazır veri birbirinden ayrılmış oldu.

- **Bronze Katman:** Kafka'dan gelen ham JSON mesajlarının ilk haliyle tutulduğu katmandır. Veriyi bozmadığımız için hata olduğunda geri dönüp kontrol edebiliyoruz.
- **Silver Katman:** Bronze'dan gelen verinin temizlenmiş halidir. Eksik tarih veya şehir bilgisi olan kayıtlar çıkarıldı, sıcaklık için mantıksız değerler elendi ve veri tipleri düzenlendi.
- **Gold Katman:** Analiz ve makine öğrenmesi için hazırlanan son katmandır. Burada aylık özetler, model sonuçları ve eğitimde kullanılacak feature tabloları üretildi.

Gold tarafında özellikle şu tabloları kullandık:

- **ml_features:** Modellerin kullanacağı sıcaklık, gecikmeli değer, kayan ortalama, mevsim ve benzeri değişkenleri içerdi.
- **city_monthly_summary:** Şehir ve ay bazında ortalama sıcaklık, toplam yağış ve yağışlı gün sayısı gibi özetleri tuttu.
- **model_results:** Eğittiğimiz modellerin RMSE, MAE ve R2 sonuçlarını karşılaştırmak için hazırlandı.
- **climate_features:** Rüzgar kategorisi ve veri doluluk durumu gibi ek özellikleri topladığımız tablo oldu.

1.3 Kullanılan Servisler

Servis	Projede Ne İşe Yaradı ?
zookeeper	Kafka'nın düzgün çalışması için koordinasyon sağladı.
kafka	Verilerin mesaj olarak aktığı ana kuyruk sistemi oldu.
kafka-ui	Topic ve mesajları tarayıcıdan kontrol etmemizi sağladı.
producer	Parquet verisini okuyup Kafka'ya JSON olarak gönderdi.
spark	Kafka'dan gelen veriyi işledi ve Delta tablolara yazdı.
mlflow-ui	Model denemelerini ve metrikleri takip etmek için kullanıldı.
streamlit	Sonuçları interaktif dashboard şeklinde göstermek için hazırlandı.

2. Karşılaştığımız Zorluklar

2.1 Kafka Listener Ayarları

Projede ilk zorlandığımız konulardan biri Kafka bağlantısıydı. Spark ve Producer container içinden Kafka'ya bağlanırken, biz de bazen kendi bilgisayarımızdan test etmek istedik. Tek listener kullanınca bir taraf çalışırken diğer taraf sorun çıkarabiliyordu. Bu yüzden container ağı için kafka:9092, dışarıdan erişim için localhost:29092 olacak şekilde iki ayrı adres kullandık. Bu ayar yapılmayınca sistemin tamamı durmuş gibi görünebiliyor.

2.2 Producer'ın Kafka'dan Önce Başlaması

Producer bazen Kafka tam hazır olmadan çalışmaya başlıyordu. Böyle olunca bağlantı hatası alıp kapanıyordu. Bunu çözmek için Kafka servisine healthcheck ekledik ve Producer'ı Kafka sağlıklı durumdayken başlatacak şekilde ayarladık. Bu değişiklikten sonra akış daha düzenli çalıştı.

2.3 Eksik Sensör Verileri

EDA sırasında bazı sütunlarda çok yüksek oranda eksik veri olduğunu fark ettik. Özellikle rüzgar hızı, deniz seviyesi basıncı ve güneşlenme süresi gibi alanlarda eksiklik oranı bazı durumlarda %95'in üstündeydi. Bu yüzden bu alanlardan üretilen bazı feature'lar modele beklediğimiz kadar katkı vermedi. Mesela wind_category çoğu kayıt için unknown kaldı.

3. Sonuçlar

Model	RMSE (°C)	MAE (°C)	R2
Linear Regression	0.333	0.247	0.9987
GLR (Generalized LR)	0.333	0.247	0.9987
GBT	0.731	0.390	0.9939
Random Forest	0.818	0.483	0.9923
Decision Tree	0.846	0.472	0.9918

4. Genel Değerlendirme

- **Katmanlı yapı:** Bronze, Silver ve Gold ayrımı verinin daha düzenli yönetilmesini sağladı.
- **Docker kullanımı:** Servisleri container içinde çalıştırmak kurulumu ve tekrar çalıştırmayı kolaylaştırdı.
- **MLflow takibi:** Model denemelerini kaydetmek, daha sonra sonuçları karşılaştırmayı kolaylaştırdı.
- **Dashboard tarafı:** Hem Jupyter tarafında grafikler üretildi hem de Streamlit ile daha interaktif bir gösterim hazırlandı.

Genel olarak bu proje bize büyük veri tarafında sadece kod yazmanın değil, sistemin parçalarını doğru bağlamanın da önemli olduğunu gösterdi. Kafka, Spark, Delta Lake, MLflow ve Streamlit gibi araçları aynı akış içinde kullanarak uçtan uca çalışan bir yapı kurduk. Bazı teknik sorunlar ve veri kalitesi problemleri yaşadık ama bunlar da projenin öğrenme kısmını güçlendirdi. İleride daha fazla şehir verisiyle, leakage problemi giderilmiş modellerle ve daha gerçek zamanlı çalışan bir dashboard ile proje daha güçlü hale getirilebilir.